

UNIVERSIDADE DO OESTE DE SANTA CATARINA – UNOESC
CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

JOÃO VITOR BERNARDON SCHWEIKART

LINHA VIVA: RELATÓRIO TÉCNICO DE DESENVOLVIMENTO DE APLICATIVO
MÓVEL PARA ACOMPANHAMENTO DO TRANSPORTE COLETIVO DE
CHAPECÓ – SC

CHAPECÓ – SC
2026

1 INTRODUÇÃO

Este relatório descreve a implementação do Linha Viva, aplicativo móvel para acompanhamento do transporte coletivo urbano de Chapecó (SC), cuja concepção, requisitos e protótipo foram definidos na Atividade 1 da disciplina. O objetivo da aplicação é reduzir a incerteza sobre o tempo de espera do passageiro, respondendo em poucos toques a três perguntas: qual linha atende ao destino, por onde ela passa e quanto tempo falta para a próxima passagem.

O escopo desta etapa foi transformar o protótipo em um aplicativo Android funcional, compilável e executável, mantendo as decisões funcionais, arquiteturais e de experiência de uso já estabelecidas. O documento concentra-se no processo de desenvolvimento, nas escolhas técnicas e nas soluções adotadas diante das restrições encontradas.

Todos os dados utilizados são fictícios e servem exclusivamente à demonstração acadêmica. A geografia e os bairros de Chapecó são reais, mas linhas, itinerários, horários, tarifas e contatos foram criados para o trabalho e não representam a operação da Auto Viação Chapecó.

2 TECNOLOGIAS E AMBIENTE

A aplicação foi desenvolvida de forma nativa para Android, em Java, com Gradle como sistema de compilação. A versão mínima suportada é a API 26 (Android 8.0), conforme o requisito não funcional RNF08, o que abrange aparelhos de entrada ainda em uso pelo público-alvo descrito na Atividade 1. Cabe registrar que a Atividade 1 previa Kotlin; a implementação foi conduzida em Java por exigência desta etapa, preservando as demais decisões de projeto.

Quadro 1 – Tecnologias empregadas e respectivas funções

Tecnologia	Versão	Função no projeto
Java	17	Linguagem do aplicativo Android
Gradle / AGP	8.2 / 8.2.2	Compilação, dependências e empacotamento
Room	2.6.1	Banco de dados local e consultas observáveis
Retrofit + Gson	2.9.0	Cliente REST e desserialização de JSON
OkHttp	4.11.0	Transporte HTTP e registro de requisições
Lifecycle (ViewModel, LiveData)	2.7.0	Padrão MVVM e ciclo de vida
Material Components	1.11.0	Componentes de interface

Tecnologia	Versão	Função no projeto
osmdroid	6.1.18	Mapa baseado em OpenStreetMap
ZXing	4.3.0	Leitura do QR Code do ponto de parada
Node.js	20	API REST de demonstração
JUnit	4.13.2	Testes unitários

O ambiente de desenvolvimento foi montado com o Android SDK em linha de comando, plataforma 34 e ferramentas de compilação 34.0.0, sobre JDK 17. O projeto inclui o Gradle Wrapper, de modo que a compilação não depende de instalação prévia do Gradle na máquina.

3 ARQUITETURA DA SOLUÇÃO

A aplicação foi organizada no padrão MVVM, com quatro camadas de responsabilidade bem delimitada. A regra central que orienta toda a arquitetura é que o banco local constitui a única fonte de verdade da interface: as telas observam exclusivamente o Room, e a rede apenas alimenta o Room. Essa inversão é o que sustenta a estratégia offline-first prevista nos requisitos RNF04 e RNF05.

3.1 Camada de interface

As Activities e os Fragments limitam-se a inflar layouts, observar LiveData e encaminhar eventos do usuário. Nenhuma delas consulta rede ou banco diretamente. A navegação principal usa uma barra inferior fixa com quatro destinos, e os fragmentos são alternados por show/hide, preservando o estado de rolagem e do mapa entre as abas. As telas de detalhe abrem como Activities sobre esses destinos, sempre com retorno explícito, conforme o fluxo do protótipo.

3.2 ViewModels

Os sete ViewModels concentram filtros, buscas, cálculo de previsões e os estados de carregamento, erro, vazio e offline. Composições que dependem de múltiplas fontes usam MediatorLiveData; quando o resultado exige leitura do banco, o processamento é despachado a um executor de segundo plano e devolvido por postValue, nunca bloqueando a thread principal.

3.3 Repositórios e persistência local

Quatro repositórios isolam as fontes de dados das camadas superiores, apoiados por três colaboradores: Sincronizador, responsável pela carga a partir da API; Cargalnicial, que popula o banco com os dados embarcados; e ChamadaApi, que centraliza a execução das requisições e o tratamento de falhas. O banco possui nove entidades: linha, ponto, itinerário, horário, previsão por ponto, previsão por linha, favorito, aviso e informação. Elas são acessadas por oito DAOs que devolvem LiveData, garantindo que qualquer gravação atualize a interface automaticamente.

3.4 Camada de rede

A comunicação REST está inteiramente contida no pacote data/remote, composto pela interface Retrofit, pelo construtor do cliente e por onze DTOs. A conversão entre o formato da API e o modelo local ocorre em uma única classe, o Mapeador, que funciona como a fronteira do sistema. Essa separação atende ao requisito de permitir a substituição futura por uma API real: seriam necessárias alterações apenas nesses três pontos, sem impacto sobre ViewModels, telas ou banco local.

Atendendo ao conceito estudado na disciplina, nenhuma requisição de rede executa na thread principal. As chamadas são síncronas, executadas por um ExecutorService com três threads, e o resultado retorna à interface por LiveData, evitando a exceção NetworkOnMainThreadException e mantendo a interface responsiva.

4 API REST DE DEMONSTRAÇÃO

4.1 Justificativa

A Atividade 1 registrou, como dependência crítica do projeto, a ausência de garantia de uma API pública de rastreamento da frota de Chapecó. Para que o aplicativo fosse executável e demonstrável, optou-se por desenvolver uma API própria, com dados fictícios, reproduzindo a arquitetura cliente-servidor de um caso real. A alternativa de embarcar todos os dados no aplicativo foi descartada por descaracterizar o requisito de consumo de API REST.

O servidor foi escrito em Node.js sem qualquer dependência externa, utilizando apenas o módulo HTTP nativo. A decisão elimina a etapa de instalação de pacotes e reduz o risco de o ambiente de avaliação não conseguir executá-lo: basta um comando para colocá-lo no ar.

4.2 Interface do serviço

Foram implementados dezessete endpoints sob o prefixo /api/v1. Toda resposta de coleção é encapsulada em um envelope que declara a data de geração e a natureza fictícia dos dados, de modo que a origem da informação acompanhe o próprio conteúdo.

Quadro 2 – Principais endpoints da API de demonstração

Endpoint	Finalidade
GET /versao	Assinatura do catálogo, base da sincronização incremental
GET /linhas	Relação das linhas, com sentidos e intervalos
GET /linhas/previsoes	Próxima partida de cada linha
GET /linhas/{id}/itinerario	Sequência de pontos de um sentido
GET /linhas/{id}/horarios	Horários programados por tipo de dia
GET /linhas/{id}/veiculo	Posição simulada do veículo em circulação
GET /itinerarios e /horarios	Carga em lote, usada na sincronização
GET /pontos	Pontos, com filtro por proximidade ou por texto
GET /pontos/{id}/previsoes	Previsões de chegada em um ponto
GET /avisos e /informacoes	Avisos de operação, tarifas, terminais e contatos

4.3 Simulação de previsões e de posição

Como não há rastreamento real, previsões e posições são calculadas a partir da tabela de horários programados e do relógio do servidor. O serviço identifica quais viagens estariam em curso, interpola a posição do veículo ao longo do itinerário e aplica um desvio determinístico por viagem, o que produz variação realista sem tornar o resultado imprevisível entre execuções.

Três medidas garantem que o dado simulado não seja apresentado como real: cada resposta carrega o campo origem com o valor SIMULADO ou HORARIO_PROGRAMADO; o envelope inclui uma observação textual sobre a estimativa; e as telas de acompanhamento e de ponto exibem, em rodapé fixo, o aviso de que a posição é estimada e depende de integração futura com os dados da empresa.

O conjunto de demonstração contempla oito linhas, trinta e sete pontos, noventa e oito trechos de itinerário, mil cento e quinze horários, quatro avisos de operação e dez registros de informações úteis, volume suficiente para exercitar busca, filtros, paginação visual e todos os estados da interface.

5 IMPLEMENTAÇÃO DAS TELAS

Foram implementadas as oito telas previstas no protótipo, preservando a identidade visual definida na Atividade 1: azul institucional como cor primária, verde reservado exclusivamente à informação em tempo real e amarelo para avisos operacionais e estado offline.

Quadro 3 – Telas implementadas e requisitos atendidos

Tela	Conteúdo principal	Requisitos
Splash	Carga local e tentativa de sincronização	RNF01, RNF05
Mapa	GPS, pontos próximos, traçados, busca e filtros	RF02, RF04, RF06
Linhas	Listagem, busca e quatro filtros	RF03, RF04, RF08
Detalhe da linha	Itinerário, horários e mapa, por sentido	RF05, RF08
Tempo real	Veículo no mapa, estimativa e aviso	RF07, RF09, RF14
Ponto de parada	Partidas, infraestrutura e QR Code	RF06, RF13, RF14
Favoritos	Linhas, pontos, avisos e atalhos	RF08, RF11, RF12
Mais	Serviços, servidor de dados e sincronização	RF12

6 PRINCIPAIS DECISÕES DE PROJETO

6.1 Mapa sem chave de API

A escolha da biblioteca de mapa considerou o requisito de que o projeto compilasse e executasse sem configuração adicional. O Google Maps SDK exigiria que o avaliador criasse uma chave no Google Cloud e a inserisse no projeto; sem ela, o mapa seria exibido em branco. Optou-se pelo osmdroid, sobre dados do OpenStreetMap, que dispensa chave, mantém cache de tiles em disco — o que contribui para o funcionamento offline — e apresenta o traçado real das ruas de Chapecó. O custo dessa decisão é a perda de recursos próprios do ecossistema Google, nenhum deles necessário ao escopo atual.

6.2 Localização pela API da plataforma

Para a obtenção da posição do usuário optou-se pelo LocationManager do próprio sistema, em vez do provedor fundido dos serviços do Google. A decisão elimina uma dependência de Play Services, que não está presente em todas as imagens de emulador, e evita falhas de execução em ambientes de avaliação. O consumo de bateria é controlado por uma implementação de LiveData que solicita atualizações apenas

enquanto alguma tela observa a localização e as encerra quando o último observador é removido, atendendo diretamente ao RNF02.

6.3 Concorrência com executores e LiveData

Sem corrotinas disponíveis em Java, a concorrência foi resolvida com um `ExecutorService` compartilhado e um executor para a thread principal, encapsulados em uma classe utilitária. As chamadas Retrofit são síncronas e sempre despachadas a esse executor, o que torna explícito o ponto de troca de thread e simplifica o rastreamento de falhas, em comparação com o uso de callbacks assíncronos espalhados pelos repositórios.

6.4 Sincronização incremental por assinatura

O RNF04 determina sincronizar apenas as diferenças e o RNF06 exige volumes reduzidos de dados. Como solução, a API expõe um endpoint que devolve uma assinatura do catálogo estático. O aplicativo compara essa assinatura com a guardada localmente e, quando coincidem, dispensa o download de linhas, pontos, itinerários e horários, baixando somente o conteúdo volátil: avisos, informações e previsões. Quando o catálogo precisa ser rebaixado, os endpoints em lote substituem dezenas de requisições individuais por quatro.

7 ESTRATÉGIA DE FUNCIONAMENTO OFFLINE

O funcionamento sem conexão foi tratado como característica estrutural, e não como tratamento de exceção. A solução se apoia em três camadas complementares.

a) Carga inicial embarcada. Arquivos JSON de aproximadamente cento e cinco kilobytes acompanham o aplicativo e populam o banco na primeira execução. O aplicativo abre útil mesmo que nunca tenha alcançado a API.

b) Banco local como fonte de verdade. Como a interface observa apenas o Room, a indisponibilidade da rede não produz tela de erro: produz ausência de atualização. Gravações ocorrem em transação e somente quando a resposta chega completa, de modo que uma falha jamais apaga dados anteriores.

c) Estado de conexão explícito. Uma classe compartilhada combina dois sinais — o transporte de rede informado pelo sistema e o resultado real das chamadas à API. Ter Wi-Fi e não alcançar o servidor é, para o usuário, equivalente a estar offline, e as duas situações exibem a mesma faixa âmbar com a data da última atualização.

Um cuidado adicional refere-se à validade da informação apresentada. Previsões guardadas em cache não são reexibidas como atuais: os minutos restantes são sempre recalculados a partir do horário e, quando a previsão armazenada já venceu, o cálculo recorre à tabela de horários programados, disponível localmente. Assim, na ausência de dado em tempo real, a interface degrada de forma controlada para o horário previsto, sem apresentar erro ao usuário, exatamente como determina o RNF12.

8 VALIDAÇÃO E CORREÇÕES

8.1 Procedimentos adotados

A validação combinou quatro procedimentos. O primeiro foi a compilação completa, que produziu um pacote de instalação de sete megabytes. O segundo foi a execução de quinze testes unitários, dos quais dez verificam que os objetos de transferência de dados permanecem compatíveis com respostas reais da API, capturadas do servidor e armazenadas como massa de teste; os demais cobrem conversões de horário, distância e tempo relativo. O terceiro foi a análise estática com o Android Lint, encerrada sem erros ou problemas fatais.

O quarto procedimento, e o mais produtivo, foi a execução do aplicativo em emulador com Android 14 (API 34). As oito telas foram percorridas, a integração REST foi confirmada por registro de tráfego HTTP e o modo offline foi verificado com o servidor derrubado e a rede desativada. Essa etapa revelou sete defeitos que a compilação e a análise estática não seriam capazes de detectar.

8.2 Defeitos identificados e soluções

Quadro 4 – Defeitos encontrados em execução e respectivas soluções

Defeito observado	Solução adotada
A faixa de modo offline não aparecia em telas criadas antes da queda da conexão.	O monitor publicava o estado de uma rede isolada. Passou a reavaliar a rede ativa em todos os retornos do sistema.
Previsões vencidas exibiam “Sem previsão” em vez do horário programado.	Criada uma rotina única de cálculo que recorre à tabela de horários quando a previsão em cache já passou.
A construção do estado de conexão em thread de background provocava encerramento na abertura.	A instância passou a ser criada na thread principal, com publicação assíncrona de valores.
Duas partidas da mesma linha no mesmo ponto eram indistinguíveis.	O sentido da viagem passou a compor a descrição de cada previsão.
O seletor de sentido ficava ilegível quando selecionado.	Substituídas as cores fixas por seletores de estado, garantindo contraste nas duas situações.

Defeito observado	Solução adotada
Itinerário e horários eram baixados duas vezes ao abrir uma linha.	Removida a sincronização redundante na inicialização, mantida apenas a do retorno à tela.
A estrela de favorito colidia com o texto de previsão em nomes longos.	Ajustadas margens e largura mínima do bloco de previsão.

O terceiro defeito do quadro merece registro particular por ter sido introduzido durante a correção do primeiro: a solução do problema de conectividade passou a construir um objeto observável fora da thread principal, o que viola uma restrição do componente LiveData. O episódio evidencia a importância de reexecutar a aplicação após cada correção, e não apenas ao final do ciclo.

9 REQUISITOS ATENDIDOS

Os quatorze requisitos funcionais definidos na Atividade 1 foram implementados. Dois deles operam sobre dados simulados, e um possui alcance reduzido, conforme detalhado a seguir.

Quadro 5 – Situação dos requisitos funcionais

ID	Requisito	Situação
RF01 a RF05	Consulta sem cadastro, mapa, listagem, busca e itinerário	Implementados
RF06	Previsão de chegada em um ponto	Implementado com dados simulados
RF07	Acompanhamento do veículo em tempo real	Implementado com dados simulados
RF08	Favoritos de linhas e de pontos	Implementado
RF09	Notificação de aproximação do veículo	Implementado com a tela aberta
RF10 a RF12	Uso offline, avisos e informações úteis	Implementados
RF13	Abertura do ponto por QR Code	Implementado, com entrada manual alternativa
RF14	Acessibilidade e nível de lotação	Implementado

Quanto aos doze requisitos não funcionais, todos foram contemplados. Merecem destaque os que orientaram decisões estruturais: a persistência local e o modo offline informando a data da última atualização (RNF04 e RNF05); o uso do GPS restrito ao período de utilização ativa e a limitação de atualizações em segundo plano (RNF02 e RNF03); a redução do volume transferido por meio da sincronização incremental

(RNF06); o layout adaptável a diferentes tamanhos de tela (RNF07); e a degradação controlada na ausência de dado em tempo real (RNF12).

10 CONSIDERAÇÕES FINAIS

O projeto entregou um aplicativo Android executável que implementa os quatorze requisitos funcionais e os doze requisitos não funcionais definidos na Atividade 1, acompanhado de uma API REST própria que sustenta a demonstração. As decisões técnicas mais relevantes foram orientadas por duas prioridades: garantir que o projeto compile e execute sem configuração adicional e respeitar as restrições de bateria, memória e conectividade discutidas na disciplina.

A etapa de execução em emulador mostrou-se decisiva. Sete defeitos relevantes, entre eles a falha da sinalização de modo offline e um encerramento inesperado na abertura, somente se manifestaram com a aplicação em funcionamento. A experiência confirma que compilação bem-sucedida e análise estática limpa não substituem a verificação do comportamento real.

Como continuidade, mantêm-se as direções previstas na Atividade 1: testes de usabilidade com passageiros, busca de acordo institucional para acesso aos dados de posicionamento da frota e, em etapa posterior, avaliação da expansão para iOS.